



PHP functions 15m

As previously discussed in the topic about JavaScript, functions are the building blocks for any programming language because they allow code components that perform specific tasks (for example, a function to calculate tax on a price) to be written as reusable units of code.

Structure of a function

A PHP function is declared or defined by the reserved keyword 'function'. The function keyword is followed by the name of the function and zero or more parameters (these are the argument values that can be passed into the function at runtime if required).

The following code declares a function called **addNumbers**, which receives two parameter arguments (the values to be added together) and returns the result. Anything after the 'return' keyword is the value that is returned to the code that called the **addNumbers** function. Execution of the function ends at the 'return', so any code that appears after it would not be executed.

The test code shows two ways of calling the function. The first puts the result into a variable; the second directly echoes the result of calling the function.



[Skip to main content](#)



```
1  // function to add two numbers
2      function addNumbers($number1, $number2)
3      {
4          return $number1 + $number2;
5      };
6  // put the result into a variable
7      $sum = addNumbers(123,456);
8      echo  $sum;
9
10 // use the function directly
11      echo addNumbers(456,789) ;
```



You can also use default values for function parameters so arguments can be left out when calling the function. In this example, the second parameter has a default value of 10. If the function is called with only one argument instead of two, the default value will be used so it will be added to 10. You can have as many default values as you like, but they must start from the right-hand end of the parameter list.



[Skip to main content](#)



```
1 function addNumbers2($number1, $number2=10)
2 {
3     return $number1 + $number2;
4 };
5 // put the result into a variable – only one argument passed
6 $sum = addNumbers2(20);
7 // you should get the value 30 here
8 echo $sum;
```



You can also get a function to return multiple values, where the simplest method is to return the results as an array. Then, when you call the function, you take the returned value and put it into a list of variables.

The following code has a couple of things to note. First, just for the sake of an example, it has default values for both arguments, so you don't need to pass anything into the function. More importantly, an array is created and returned inside the function.

Note the syntax used here to create the array on the fly, using the **array** function. To retrieve the data from that returned array you use the built-in PHP **list** function. You can then access the elements of that list using the variable names.



[Skip to main content](#)



```
1 function twoResults($number1=10, $number2=20)
2 {
3     // return results as an array
4     return array("The sum is ", $number1 + $number2);
5 };
6
7 // extract from the array as a list of variables
8 list($var1, $var2) = twoResults();
9 echo $var1.$var2;
```



PHP date and time functions

PHP includes many functions related to dates and times, the complete set of which you can find on the reference page for the date and time functions. However, this section focuses mainly on the date function, which returns the current date and/or time as a formatted string because the date function is the most commonly used for date and time-related functions.

Using the date function can be a bit confusing because you must specify the format of the data that you want to access using a string of special characters. Each character relates to a different component of the current date or time. Note the examples in the following code.



[Skip to main content](#)



```
1 echo "Today is ".date("Y/m/d");  
2 echo "\nToday is ".date("l");  
3 echo "\nThe time is ".date("h:i:sa");  
4 echo "\nThere are ".date("t")." days in ".date("M");
```



Can you see how using a different string of special characters returns a different component of the current date and time?

The output we got for this code is shown below. Your output will have a similar format to ours. However, your time and dates will be different because your current date will differ from ours (we ran this code on 23 May 2022 at 10.04 am).

Output:

```
Today is 2022/05/23  
Today is Monday  
The time is 10:04:26am  
There are 31 days in May
```

In the first example, the string **Y/m/d** causes the current date to be formatted using the full year (uppercase **Y**), the month number (lowercase **m**) and the day number (lowercase **d**). The forward slash characters are not special date characters; they are just separators added between the values. The same goes for the colons in the third example. From the other examples, you can see that lowercase **i** gives the full name of the day, uppercase **M** the full name of the month, and so on.

Since dates are also used extensively with databases, it would be a good idea to start creating yourself a cheat sheet of some of the more useful date functions and the data formatting strings. This will certainly come in handy with Topic 10, which uses PHP with

[Skip to main content](#)

databases.

For a full table of the characters that can be used with the date function, see the PHP manual on date/time formats (The PHP Documentation Group, n.d.).

Setting dates and times

When used with one argument, the date function lets us access the system date and time. But what if you want to create a date and/or time that is different from the current one? In that case, you can use the `mktime` function, which returns the Unix timestamp for a date. The syntax for using `mktime` is as follows:

```
mktime(hour, minute, second, month, day, year)
```

Arguments may be left out in order from right to left. Any arguments not specified will be set to the current date and time values. You must provide at least the first argument (the hour).

The `mktime` function returns a timestamp that can be formatted by passing it as the optional second argument to the date function. In this example, a timestamp is created and echoed to the screen. Then it is formatted by the date function, and the day of the week is displayed from that date.

The time zone setting is also included here, using the `date_default_timezone_set` function, to ensure the calendar works in the New Zealand time zone; otherwise, it might be 12 hours out. The example below will set the current time to 11.00 am, but since the values have not been entered for the month, day or year, it will default to the current date.



[Skip to main content](#)



```
1 // set the default timezone for your region. date_default_timezone_set('N
  Z');
2 $new_date=mktime(11,0,0); // set the time - the date will default to the cur
  rent date
3 echo "Created date is " . date("Y-m-d h:i:sa", $new_date); // timestamp
4 echo "\n".date('l',$new_date); // day of the week
5 // Check if Daylight Saving Time is active
6 if (date("I",$new_date) == "1") {
7     echo "Daylight Saving Time is active";
8 }
9 else {
10     echo "Daylight Saving Time is not active";
11 }
```



Converting date strings to arrays

The functions you have looked at so far create a string representing a date and/or time. In contrast, `date_parse` is a useful function that can take a date string, provided in a suitable format like the example below, and convert it into an array. This is useful if you want to have detailed information about a specified date.

```
<?php

var_dump(date_parse("2022-12-12 10:00:00"));

?>
```

This will have the output:

[Skip to main content](#)

```
array(12) { ["year"]=> int(2022) ["month"]=> int(12) ["day"]=> int(12) ["hour"]=> int(10)
["minute"]=> int(0) ["second"]=> int(0) ["fraction"]=> float(0) ["warning_count"]=>
int(0) ["warnings"]=> array(0) { } ["error_count"]=> int(0) ["errors"]=> array(0) { }
["tyig"is_localtime"]=> bool(false) }
```

NOTE: Always include the **date_default_timezone_set** function at the top of your index.php file to ensure your web application will always process dates against your chosen time zone. More complex time zone handling would be required if your web application is to be internationally aware.



Thoughts on this page?

